

A Factorized Distribution Algorithm Using Single Connected Bayesian Networks

Alberto Ochoa^{1,2}, Heinz Muehlenbein¹, and Marta Soto²

¹ Real World Computing Partnership
Theoretical Foundation GMD Laboratory
GMD-Forschungszentrum Informationstechnik
53754 St. Augustin, Germany
muehlenbein@gmd.de

² Institute of Cybernetics, Mathematics and Physics
Center of Mathematics and Theoretical Physics
ICIMAF, CP 10400, La Habana, Cuba
{ochoa,mrosa}@cidet.icmf.inf.cu

Abstract. Single connected Factorized Distribution Algorithms (FDA-SC) use factorizations of the joint distribution, which are trees, forests or polytrees. At each stage of the evolution they build a polytree from which new points are sampled. We study empirically the relation between the accuracy of the learned model and the quality of the new search points generated. We show that a change of the learned model before sampling might reduce the population size requirements of sampling.

1 Introduction

In ([10],[11]) we have investigated the use of search distributions in evolutionary algorithms. Search distributions generalize the concept of creating new points by genetic operators working on strings. There exist many theoretical advantages for this approach. The only difficulty is to find a good search distribution suited for optimization. We have shown that for Additively Decomposed Functions (ADF) a search distribution can be computed which is factorized into a product of marginal and conditional distributions. The factorization is derived from the structure of the function. For the corresponding algorithm, the Factorized Distribution Algorithm FDA, convergence to the optima could be shown [11, 13].

The computational complexity of the algorithm depends on the number of variables needed to specify the distribution. The factorization derived from the structure of the function might use more variables than necessary. There have been techniques developed in the theory of graphical models which learn a factorization from the data. A number of algorithms use this approach ([12, 14, 17, 6]). They do not restrict the structure of the search distributions, but work with fairly general Bayesian networks. In order to reduce the complexity of the network, the number of parents has to be restricted.

Another possibility to reduce the complexity of the network is to restrict the allowed networks to a subclass of Bayesian networks. Our single connected Factorized Distribution Algorithm (FDA-SC) uses polytree factorizations of the distribution of the best point selected at each generation. We have investigated one member of this class, the so called Polytree Approximation Distribution Algorithm (PADA) in [18]. We have found that this algorithm performs very well on a wide range of functions hard to optimize for a genetic algorithm.

In this paper we continue to explore the usage of polytrees. The outline of the paper is as follows. Section 2 presents the Polytree Approximation Distribution Algorithm (PADA) together with some background material. Section 3 presents our ideas about the transformation of the learned polytree. It introduces a sampling operator which implements one of the possible transformations. Then we present our test functions and discuss the numerical results.

2 The Polytree Approximation Distribution Algorithm

2.1 Polytree Bayesian Networks

Research on Factorized Distribution Algorithms is based on two major fields: Bayesian networks and Evolutionary Computation. A Bayesian network is a directed acyclic graph (DAG) containing nodes, representing the variables, and arcs, representing probabilistic dependencies among nodes.

Bayesian networks can be grouped according to their connection structure. An important subclass are single connected (SCG) graphs: these are graphs where no more than one (undirected) path connects every two variables. Examples of SCGs are chains, trees, forests and polytrees. The last type of graph includes the previous ones. A polytree factorization can be written as:

$$p(x) = \prod_{i=1}^n p(x_i \mid x_{i1}, x_{i2}, \dots, x_{ir})$$

where x_{ij} is the value of the j -th direct parent of variable X_i . In a polytree we require that the parents of any variable are mutually independent. Hence, for all i we have:

$$p(\pi_{x_i}) = \prod_{j=1}^r p(x_{ij})$$

where $p(\pi_{x_i})$ is the joint distribution of the parent set.

Single connected networks can be recovered from data using simple and efficient learning procedures. The approach used by our algorithm computes a list of dependence/independence weighted assertions. From this list a graph is constructed that satisfies as much as possible the assertions on the list.

2.2 The Algorithm

The Polytree Approximation Distribution Algorithm (PADA) has been designed for the class of single connected Bayesian networks [18]. The rationale behind

step 0:	$t \leftarrow 1$. Generate N points randomly.
step 1:	Get a selected set S with M points. ($M < N$), according to selection method
step 2:	Learn $p^s(x, t)$ using the LPA algorithm possibly make a suitable transformation
step 3:	Generate N points according to: $p(x, t+1) = p^s(x, t)$
step 4:	$t \leftarrow t + 1$. If the termination criteria are not met, goto to step 1 .

Table 1. PADA.

the decision of using polytrees for the Factorized Distribution Algorithm class comes from two different sources. First we have been motivated by the impressive performance of the Univariate Marginal Distribution Algorithm (UMDA) [9] even for many non-linear problems. Secondly, from the representation, inference and learning point of view, polytrees have been proved to lie in a region of computational complexity, where the tradeoff between computational cost and goodness of results is promising. For example, in [1] authors compare empirically the performance of both polytrees and general (multiply connected) Bayesian networks models learned from data. In the classification task they explored, they found that polytrees do almost the same job, despite their inability to capture high order independence relationships.

Using polytrees, we gain in efficiency and simplicity in the procedures for sampling and learning the networks. The price we have to pay for using polytrees is a less expressive power. The kind of dependency relations that may be represented, is more restricted for polytrees than for general multiply connected networks.

The Polytree Approximation Distribution Algorithm (PADA) is a FDA optimization algorithm and therefore it shares with the FDA, the general structure described in table 1. In step 2 of the algorithm, PADA uses the LPA algorithm, which is introduced in the next section, to learn a polytree approximation of the probabilistic structure of the selected set of points. Once the algorithm knows the structure, it does parametric learning using Laplace estimation [7].

2.3 The Learning Algorithm LPA

The *LPA* algorithm for learning polytrees from data is a revised version of the algorithm presented in [4] and used in the preliminary version of PADA [18]. It uses a graded dependency relation that measures the degree of dependency between two variables. The basic idea is to preserve the edges representing the strongest dependency relations, but with the restriction that the structure must be singly connected. To do this, the algorithm uses Chow-Liu's algorithm for building the skeleton (the undirected graph that supports the polytree). Chow-Liu's algorithm uses a Maximum Weight Spanning Tree (MWST) to obtain the

skeleton of the polytree, where the weight of an edge $DepG(\alpha, \beta)$, is a measure of the dependency degree between the linked variables. In our current implementation, we use the Kullback-Leibler cross entropy measure.

$$\begin{aligned} Dep(x, y) &= \sum p(x, y) \text{Ln} \left(\frac{p(x, y)}{p(x) * p(y)} \right) \\ Dep(x, y \mid z) &= \sum p(x, y, z) \text{Ln} \left(\frac{p(x, y, z) * p(z)}{p(x, y) * p(x, z)} \right) \\ DepG(x, y) &= \min(Dep(x, y), Dep(x, y \mid z)), z \in U \end{aligned}$$

Therefore, we can use marginal $Dep(x, y)$, conditional $Dep(x, y \mid z)$ and global $DepG(x, y)$ dependency measures. Once we have constructed the skeleton a procedure tries to direct the edges of the skeleton by using the following scheme: in a head to head pattern $\alpha \rightarrow \gamma \leftarrow \beta$, the instantiation of the head to head node γ should normally increase the degree of dependency between the variables α and β , whereas in a non head to head pattern (such as $\alpha \leftarrow \gamma \rightarrow \beta$, $\alpha \rightarrow \gamma \rightarrow \beta$ or $\alpha \leftarrow \gamma \leftarrow \beta$), the instantiation of the middle node γ should produce the opposite effect, decreasing the degree of dependency between α and β . The idea is to compare the degree of dependency between α and β after the instantiation of γ with the degree of dependency between α and β before the instantiation of γ .

Edge Orientation: For each subgraph $\alpha - \gamma - \beta \in \text{skeleton}$, if $Dep(\alpha, \beta \mid \gamma) > Dep(\alpha, \beta)$ then $\alpha \rightarrow \gamma \leftarrow \beta$

Edges that are not oriented after the above test, are directed at random without introducing new head to head connections. In table 2 the learning algorithm is shown. In step 4, the constraint on the number of edges guarantees that the skeleton is kept without cycles. The combination of the two measures is intended to give a global dependency measure.

The values ϵ_0 and ϵ_1 used in steps 1 and 2, are thresholds, which means that edges with values below them are considered independent. In the usual Bayesian networks applications, they are simply small positive numbers. However, in optimization, under certain circumstances, the performance of the algorithm could be sensible to even relative small departures from their true values. Both values depend on the population size. For large population sizes they are almost the same. We can easily calculate these values in an uniform random population.

The general scheme given in table 2 can be easily modified to accommodate different variants of the algorithm. The resulting algorithms have either quadratic or cubic complexity on the number of independence tests. Also, they can output trees or polytrees.

3 Transformation of the Learned Graph

The idea of transforming the learned graph before sampling, was suggested in [18]. There an optional pruning step was considered in the original scheme of PADA. Here we present a more general solution. The resulting algorithm we call FDA-SC.

0:	Start with an empty graph G and one empty list L .
1:	for every pair of nodes $\alpha, \beta \in U$ do 1.1 Compute $Dep(\alpha, \beta \mid \emptyset)$. 1.2 If $Dep(\alpha, \beta \mid \emptyset) > \epsilon_0$, insert the edge $\langle \alpha, \beta \rangle$ in L .
2:	for every $\langle \alpha, \beta \rangle$ in L , $\gamma \in U$ do 2.1 Compute $Dep(\alpha, \beta \mid \gamma)$ 2.2 If $Dep(\alpha, \beta \mid \gamma) < \epsilon_1$ remove the edge
3:	for every $\langle \alpha, \beta \rangle$ in L , $\gamma \in U$ compute $DepG(\alpha, \beta)$. Rank L in decreasing order of $DepG$.
4:	repeat Add to G the next edge in L , with the largest $DepG$, until a maximum of $n - 1$ edges, unless it creates a cycle.
5:	for each subgraph $\alpha - \gamma - \beta \in G$ do If $Dep(\alpha, \beta \mid \gamma) > Dep(\alpha, \beta \mid \emptyset)$, $\alpha - \gamma - \beta \Rightarrow \alpha \rightarrow \gamma \leftarrow \beta$.
6:	Direct the remaining edges without introducing new head to head connections.

Table 2. The learning algorithm *LPA*.

The question is : Once we have learned the corresponding graph of a given selected set of points, can we modify it, in order to get a better graph for the sampling process?

A better graph from the optimization point of view, could mean for example a graph which generates points with a larger response to selection [9], keeping the genotype variance relatively high. Also, a better graph could mean a graph which requires less points to estimate its parameters reliably. In the following we shall assume the last interpretation.

The sample size necessary to estimate correctly the parameters of a Bayesian network, is exponential in the maximum number of parents the network has. Therefore, the sample size is highly sensitive to this number. But it is known that different graphs can perform almost the same when they are used for a sampling distributions. For this reason we propose to use a restricted model class.

4 Reducing the Computational Complexity of Sampling

To the best of our knowledge, all experimental algorithms based on Bayesian networks use the same sampling algorithm. It is a Monte Carlo simulation method introduced in [8]. The method is called Probabilistic Logic Sampling (PLS). Given an ancestral ordering of all the variables (parents go before children), the

method samples X_i using $P(X_i | \pi_{X_i})$. One obvious problem of this method, is that it will require an exponential number of points in the number of parents, to estimate the conditional probabilities correctly.

Both in the process of learning and sampling, algorithms that learn general Bayesian networks need a population size, which is exponential in the number of parents. This is important to get reliable estimates of the conditional probabilities. FDA-SC is in a different situation.

FDA-SC has the same computational complexity as other new developed Bayesian optimization algorithms: $O(n^3)$. But it can even be modified to scale only as $O(n^2)$. The computational complexity is determined by the number of dependency tests or computed marginals.

In addition, the computational complexity of FDA-SC is not exponential in the number of parents. FDA-SC uses marginals of at most order 3. This brings us an additional advantage related to reliability: both values $Dep(\alpha, \beta | \gamma)$ and $Dep(\alpha, \beta)$ can be computed much more reliably than conditional probabilities involving many variables in the conditioning set. This fact allows us to use smaller data set as the inputs for the learning algorithm. This means less requirements for the population size.

The *LPA* algorithm used by FDA-SC, deals only with conditional probabilities of first order. However, the created polytree can have nodes with more than two parents. In order to keep the advantage of FDA-SC regarding its bounded complexity in the learning phase, we have to introduce a sampling operator that uses up to second order conditional probabilities. In other words, we should force FDA-SC to use up to third order distributions only.

4.1 A New Sampling Operator for Polytrees

The sampling method is also based on PLS. Let us assume, without loss of generality, that $\pi_{X_0} = \{X_1, X_2, \dots, X_r\}$. Hence, once the parents are instantiated, the PLS method will sample X_0 with probability $P(X_0 | \pi_{X_0})$. We now transform the subgraph $\{X_0, X_1, X_2, \dots, X_r\}$, connecting the parents in the following way: $X_2 - X_3 - X_4 - \dots - X_r$, and then removing all the directions from the graph. The resulting subgraph is called junction tree, and the transformation is similar to moralization [15].

Each triplet X_0, X_i, X_{i+1} is a clique of the subgraph. In junction trees, if we know the values of a clique, we can sample the variables of any clique overlapping with it, conditionally on the overlapping variables. Because, the parents variables are instantiated we use the above schema, to implement a simple simulation rejection method.

step 0:	$x_0 \leftarrow p(x_0 x_1, x_2)$
step 1:	for $i = 3$ to r $z \leftarrow p(x_i x_0 x_{i-1})$ if $not(z = x_i)$ goto step 0
step 2:	return x_0

For many practical (sparse) problems this algorithm will converge immediately. However, the simulation time may rise with the number of parents. It is possible to devise several stopping and estimation criteria, but we will not further elaborate on this.

5 The Test Functions

Now we present the set of additive decomposable functions (ADF), that will be used in our experiments.

1. The deceptive function of order k , F_{decK} , is defined as follows. u denotes the number of 1s in the string.

$$f_{decK} \begin{cases} k-1 & \text{for } u = 0 \\ \dots & \\ k-i-1 & \text{for } u = i \\ \dots & \\ k & \text{for } u = k \end{cases}$$

Then F_{decK} is separable function of subset size k ,

$$F_{decK} = \sum_{i=1}^l f_{decK}(x_{Ki-K+1}, \dots, x_{Ki})$$

with $n = k * l$.

2. The function $F_{IsoChain}$, is an ADF with a chain-like structure. It is defined as follows:

u	0	1	2	3
Iso_1	l	0	0	$l-1$
Iso_2	0	0	0	l

$$F_{IsoChain} = \sum_{i=1}^{l-1} Iso_1(x_{2i-1}, x_{2i}, x_{2i+1}) + Iso_2(x_{2l-1}, x_{2l}, x_{2l+1})$$

with $n = 2 * l + 1$. The global optimum is $(1, 1, \dots, 1)$ with value $l(l-1)+1$. This optimum is triggered by Iso_2 . It is very isolated. Six strings with leading zeroes give the second best value of $l(l-1)$. These points are far away in Hamming distance from the optimum.

3. The next test function is like $IsoChain$, but defined on a grid of size $n = m * m$. The peak function $IsoT_2$ is used at the upper left corner of the torus. Let u denotes the number of 1s in a string.

u	0	1	2	3	4	5
$IsoT_1$	m	0	0	0	0	$m-1$
$IsoT_2$	0	0	0	0	0	m^2

$$F_{IsoTorus} = IsoT_2(x_{1-m+n}, x_{1-1+m}, x_1, x_2, x_{1+m}) \\ + \sum_{i=2}^n IsoT_1(x_{up}, x_{left}, x_i, x_{right}, x_{down})$$

where x_{up} , etc, is defined as the appropriate neighbor, wrapping around. This function is difficult to optimize. The best and second best strings have values $m^3 - m + 1$ and $m^3 - m$.

6 Numerical Results

In the experiments we compare the performance of three variants of FDA-SC:

1. using the new sampling method of section 4.1 (*Msamp*)
2. using the PLS sampling defined by the complete polytree (*Nsamp*)
3. using PLS sampling defined by polytrees with a bounded number of parents ($adj = k$)

FDA-SC uses best-elitism, which means that all the best points (the selected set) are copied to the new generation. In the tables, τ is the truncation level and G^* shows the average generation where the optima are found. N is the population size and n the number of variables. The values are computed out of 100 run.

	<i>Msamp</i>		<i>Nsamp</i>		$adj = 2$		$adj = 3$		$adj = 4$		$N = 1000$		$N = 1400$	
τ	G^*	%	G^*	%	G^*	%	G^*	%	G^*	%	G^*	%	G^*	%
0.3	10.43	85	11.08	67	11.26	63	11.17	64	11.34	61	-	-	10.02	86
0.1	5.40	82	5.47	64	6.09	72	5.73	62	5.64	68	6.03	89	-	-

Table 3. Numerical results for Deceptive 4, 100 runs.

Table 3 presents the results for the order 4 deceptive function with 32 variables and truncation selection values of 0.1 and 0.3. The population size was set to 800. In both cases the modified sampling (*Msamp*) displays better performance than the normal one. Moreover, if we constrain the numbers of neighbors of a node the results are still worse. One interesting fact is that the generation where convergence is reached is almost the same for all cases. The last columns show the results for population sizes where the normal sampling approaches the performance of the modified one. For example, for truncation 0.3, the normal sampling needed 600 more sample points.

Table 4 presents the results for the IsoChain and IsoTorus functions. For these problems we have used a truncation threshold of 0.1. The difference in the performance of the two sampling methods is still more dramatic. Under the conditions of the experiment, the IsoChain function can not be optimized with a polytree (0 success in 100 runs), unless we modify the graph to reduce the number of parents (86 success !).

Finally, it is worth noting, that the polytree graphs for all the functions contain nodes with more than three parents, particularly in the firsts generations. The results confirm the conjectures made in section 3.

			<i>Msamp</i>		<i>Nsamp</i>		<i>adj</i> = 3	
<i>function</i>	<i>n</i>	<i>N</i>	<i>G</i> *	%	<i>G</i> *	%	<i>G</i> *	%
<i>IsoTorus</i>	64	1000	9.22	95	9	62	9	47
<i>IsoChain</i>	101	2000	11.2	86	-	0		

Table 4. Numerical results for IsoFunctions.

7 Summary and Conclusions

The computational complexity of genetic algorithms and UMDA is obviously bounded by $O(N*n)$, where N is the size of the population and n is the size of the problem. Both algorithms use search distribution which can be specified by $O(n)$ parameters. They successfully solve many multi-modal optimization problems. This shows that for many optimization problems a simple search distribution is enough. For additively decomposed functions it is possible to compute a factorized distribution so that FDA will convergence to the optima [13]. But this might be computationally very expensive. In order to find the optimum, a search distribution with less parameters might be enough. But how can we find such a distribution? If the structure of the distribution is learned from the sampled data, then we have at least a computational complexity of $O(n^3)$.

This paper makes a contribution to better understand the tradeoffs of Factorized Distribution Algorithms. We have made a distinction between the quality of a Bayesian network from a statistical point of view, and its quality from an optimization point of view. FDA algorithms need search distributions which guide the search to the optimum, they do not need to precisely estimate empirical distributions. Our algorithm uses single connected Bayesian networks. We have shown that polytrees can be easily modified before sampling. The modified structure performs better than the original polytree, given the same sample size.

References

1. S. Acid, L. M. de Campos. (1995). Approximations of causal networks by polytrees: an empirical study. *Advances in Intelligent Computing*. 149-158. B. Bouchon-Meunier, R. R. Yager, L. A. Zadeh. *Lecture Notes in Computer Science* 945. Springer Verlag, Berlin.
2. S. Baluja and S. Davies. (1997). Using Optimal Dependency-Trees for Combinatorial Optimization: Learning the structure of the Search Space, (Carnegie Mellon Report, CMU-CS-97-107, 1997a).
3. C. K. Chow, C. N. Liu. (1968). Approximating discrete probability distribution with dependence trees. *IEEE Transactions on Information Theory* 14. 462-467.
4. L. M. de Campos. (1998). Independency relationships and learning algorithms for singly connected networks. *Journal of Experimental and Theoretical Artificial Intelligence*. 511-549. (Also DECSAI- TR-960204).
5. J.S. De Bonet, C. L. Isbell, and P. Viola. (1997) MIMIC: Finding Optima by Estimating Probability Densities, in M. Jordan and Th. Petsche, eds, *Advances in Neural Information Processing Systems*, Vol. 9. 424-431.

6. R. Etxeberria, P. Larranaga. (1999). Global optimization using Bayesian networks. CIMAFA 1999, Second International Symposium on Artificial Intelligence, Adaptive Systems. 332-339.
7. I. J. Good. (1965). The estimation of probabilities. MIT Press.
8. M. Henrion. (1988). Propagating uncertainty in Bayesian networks by probabilistic logic sampling. *Uncertainty in Artificial Intelligence* 2. 317-324.
9. H. Muehlenbein. (1997). The equation for response to selection and its use for prediction. *Evolutionary Computation* 5. 303-346.
10. H. Muehlenbein, G. Pass. (1996). From recombination of genes to estimation of distribution I. Binay parameters. *Lecture Notes in Computer Science* 1411. Parallel Problem Solving from Nature-PPSN IV. 178-187.
11. H. Muehlenbein, T. Mahnig, A. Ochoa. (1999). Schemata, distributions and graphical models in evolutionary optimization. *Journal of Heuristics* 5(2). 215-247.
12. H. Muehlenbein, T. Mahnig. (1999). FDA: A scalable evolutionary algorithm for the optimization of additively decomposed functions. *Evolutionary Computation* 7(4). 353-376.
13. H. Muehlenbein, Th. Mahnig. (2000). Evolutionary Algorithms: From Recombination to Search Distributions. in: *Theoretical Aspects of Evolutionary Computing*. eds. L. Kallel, B. Naudts, A. Rogers, 137-176. Springer Verlag Berlin.
14. A. Ochoa, M. Soto, R. Santana, J. C. Madera, N. Jorge. (1999). The factorized distribution algorithm and the junction tree: a learning perspective. CIMAFA 1999, Second International Symposium on Artificial Intelligence, Adaptive Systems. 368-377.
15. J. Pearl. (1988). Probabilistic reasoning in intelligent systems: networks of plausible inference. Ronald J. Brachman (AT&T Bell Laboratories). Morgan and Kaufmann.
16. M. Pelican, H. Muehlenbein. (1999). BMDA: The Bivariate Marginal Distribution Algorithm, in R. Roy, T. Furuhashi, P. K. Chandhory eds., *Advances in Soft Computing-Engineering Design and Manufacturing*, (1999), London:Springer Verlag, 521-535.
17. M. Pelican, D. E. Goldberg, E. Cantu-Paz. (1999). BOA: The Bayesian optimization algorithm. *Proceedings of the Genetic and Evolutionary Computation Conference GECCO-99* 1. 525-532.
18. M. Soto, A. Ochoa, S. Acid, L. M. de Campos. (1999). Introducing the poly-tree approximation distribution algorithm. CIMAFA 1999, Second International Symposium on Artificial Intelligence, Adaptive Systems. 360-367.